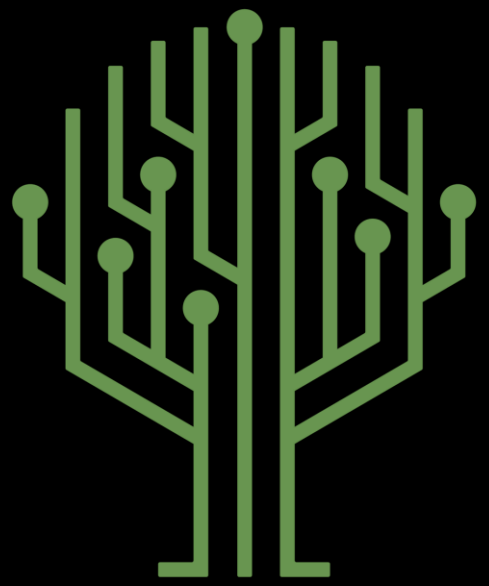
Green Pace

Security Policy Presentation

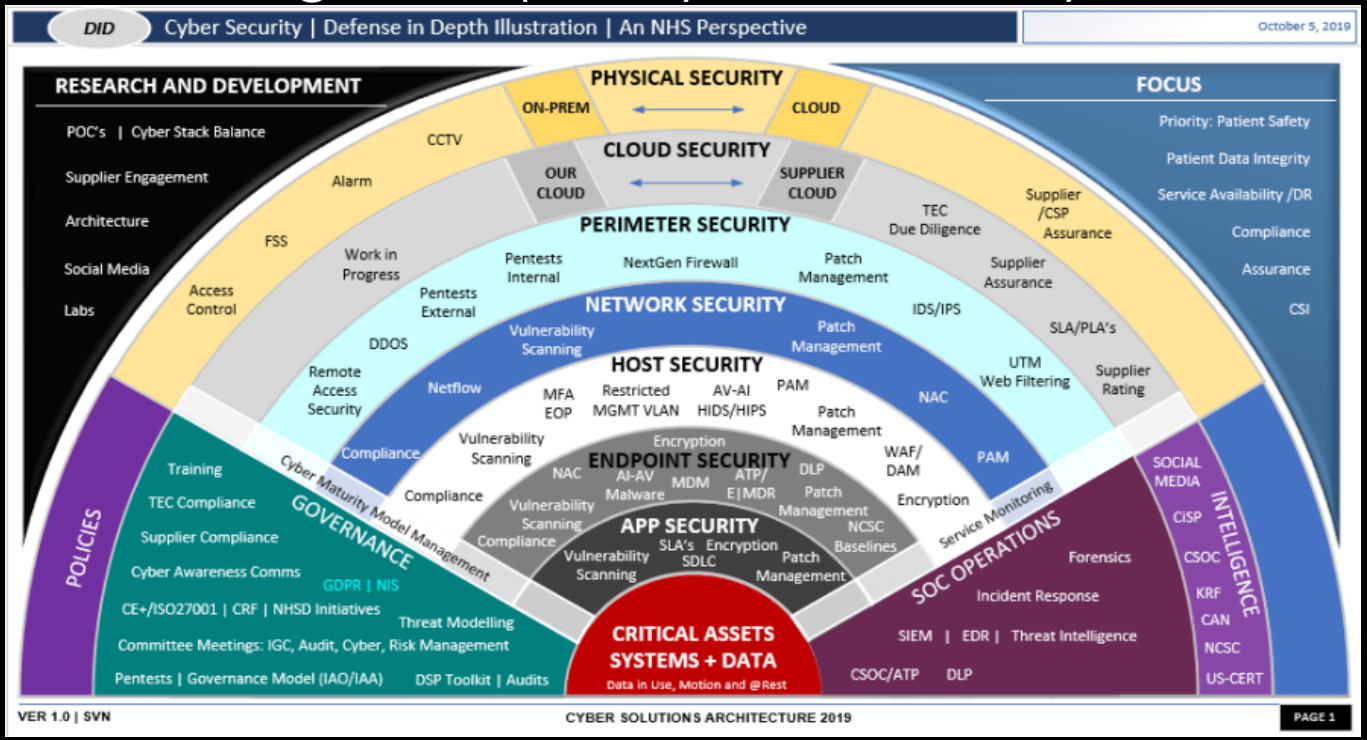
Developer: *Justin Perez*



Green Pace

OVERVIEW: DEFENSE IN DEPTH

This security policy was created to protect systems and data by establishing consistent secure practices and supporting a defense-in-depth approach through multiple layers of security controls.



Green Pace

THREATS MATRIX

Threat Matrix's group vulnerabilities on how likely they are to be exploited as well as how big of an issue they can cause.

Likely	Priority
• STD-003 – Unsafe String Functions • STD-004 – SQL Injection • STD-008 – Input Validation • STD-002 – Invalid Data Values	• STD-005 – Memory Vulnerabilities • STD-009 – Pointer Safety • STD-010 – Integer Overflow
Low priority	**Unlikely**
• STD-007 – Exception Handling • STD-001 – Incorrect Data Types	• STD-006 – Assertions

Green Pace

10 PRINCIPLES

Principles	Write a short paragraph explaining each of the 10 principles of security.
❖ **Validate Input Data**	Input validation is to ensure a user's input is clean and correct before processing the data any further to prevent malicious attacks22
❖ **Heed Compiler Warnings**	Compiler warnings usually flag potential bugs or security risk that a developer might have missed. For an example it will warn you when your scanner is left open which could consume resources or lead to vulnerability.
❖ **Architect and Design for Security Policies**	Architect and Design for Security Policies is a methodology of breaking down multiple parts of a program to design a web that hopefully will keep out intruders in case something gets breached. A common example of this could be Sever side vs client side. If the client side is breached it doesn't put ALL users at risk as the server holds most of the data and buys developers precious time to fix that vulnerability and kick out any intruders.
❖ **Keep It Simple**	Keeping it simple is a design principle that suggests simplicity should be a key goal at all stages of development as it allows for a better development experience for everyone involved. It aims to prevent over-engineering and unnecessary complexity.
❖ **Default Deny**	Default deny tells you to block ALL access to a network or system unless you specifically allow it. This is used to minimize potential vulnerabilities in an attack.
❖ **Adhere to the Principle of Least Privilege**	Adhere to the Principle of Least Privilege tells you to always do all processes using the minimal access rights needed. This could be used for safety mainly. You could think of a real-world example of this would be giving a manager of a store access to the store codes in their assigned store but not the access code to the entire company and every single store.
❖ **Sanitize Data Sent to Other Systems**	Sanitize Data Sent to Other Systems means to clean and validate all data before sending it to another program or a database as it could cause errors or leave vulnerabilities.
❖ **Practice Defense in Depth**	Practicing defense in depth means using multiple layers of security and ensuring to test it. For instance, a user could only edit their settings, however an admin can change everyone's settings. So, it's important to ensure that users' data changes when they want it to and admins can change their data as well as users' data.
❖ **Use Effective Quality Assurance Techniques**	Using Effective Quality Assurance Techniques means to do effective QA for the program in development. This could be by creating J unit test or preprogrammed inputs to test a certain program.
❖ **Adopt a Secure Coding Standard**	This is a rule that just is a warning to adopt ANY coding good coding standard. You need to have at least ONE which is commonly OWASP.

Green Pace

CODING STANDARDS

- Input Validation
- SQL Injection Prevention
- Safe String Handling
- Memory Protection
- Pointer Safety
- Integer Overflow Prevention
- Data Value Validation
- Correct Data Types
- Exception Handling
- Assertions

- These 10 are ordered from highest ranked priority to lowest ranked. Input validation is the highest due to the number of attacks that can happen while something like assertions are important still but a lot lower as they aren't commonly used for attacks.

Green Pace

ENCRYPTION POLICIES

Encryption	
Encryption at rest	Encryption at rest protects stored data such as files and databases. It ensures that even if data is accessed without permission, it cannot be read. This is important for protecting sensitive user information.
Encryption in flight	Encryption in flight protects data while it is being sent over a network. This is typically done using HTTPS or TLS. It prevents attackers from intercepting and reading data during transmission.
Encryption in use	Encryption in use protects data while it is being processed in memory. This helps prevent attackers from accessing sensitive data during execution.

Green Pace

TRIPLE-A POLICIES

Triple-A Framework	
Authentication	Authentication verifies the identity of users through login systems such as usernames and passwords. It ensures only authorized users can access the system.
Authorization	Authorization determines what a user is allowed to do after logging in. It ensures users only access data and features based on their role.
Accounting	Accounting tracks user activity such as logins, file access, and changes to the system. This helps detect suspicious behavior and supports security monitoring.

Green Pace

Unit Testing – Pointer Safety

What is being Tested

- Testing pointer initializer

How the test works

- Checks if pointers are initialized before use

What Vulnerability it detects

- Prevents crashes and unsafe memory access

How to improve

- Add runtime memory analysis tools

Green Pace

Unit Testing – Buffer Overflow

What is being Tested

- Testing buffer size limits

How the test works

- Input oversized data into buffers
- Checks if program handles safely

What Vulnerability it detects

- Prevents buffer overflow attacks

How to improve

- Use tools like Valgrind
- Add Stress Testing

Green Pace

Unit Testing – SQL Injection

What is being Tested

- Testing for query safety

How the test works

- Inputs malicious SQL Strings
- Verifies prepared statements are used

What Vulnerability it detects

- Prevents SQL injection attacks

How to improve

- Test more complex attack patterns
- Add automated database security test

Green Pace

Unit Testing – Input Validation

What is being Tested

- Testing for User Input Validation

How the test works

- Inputs Valid and invalid Values
- Checks if program rejects bad input

What Vulnerability it detects

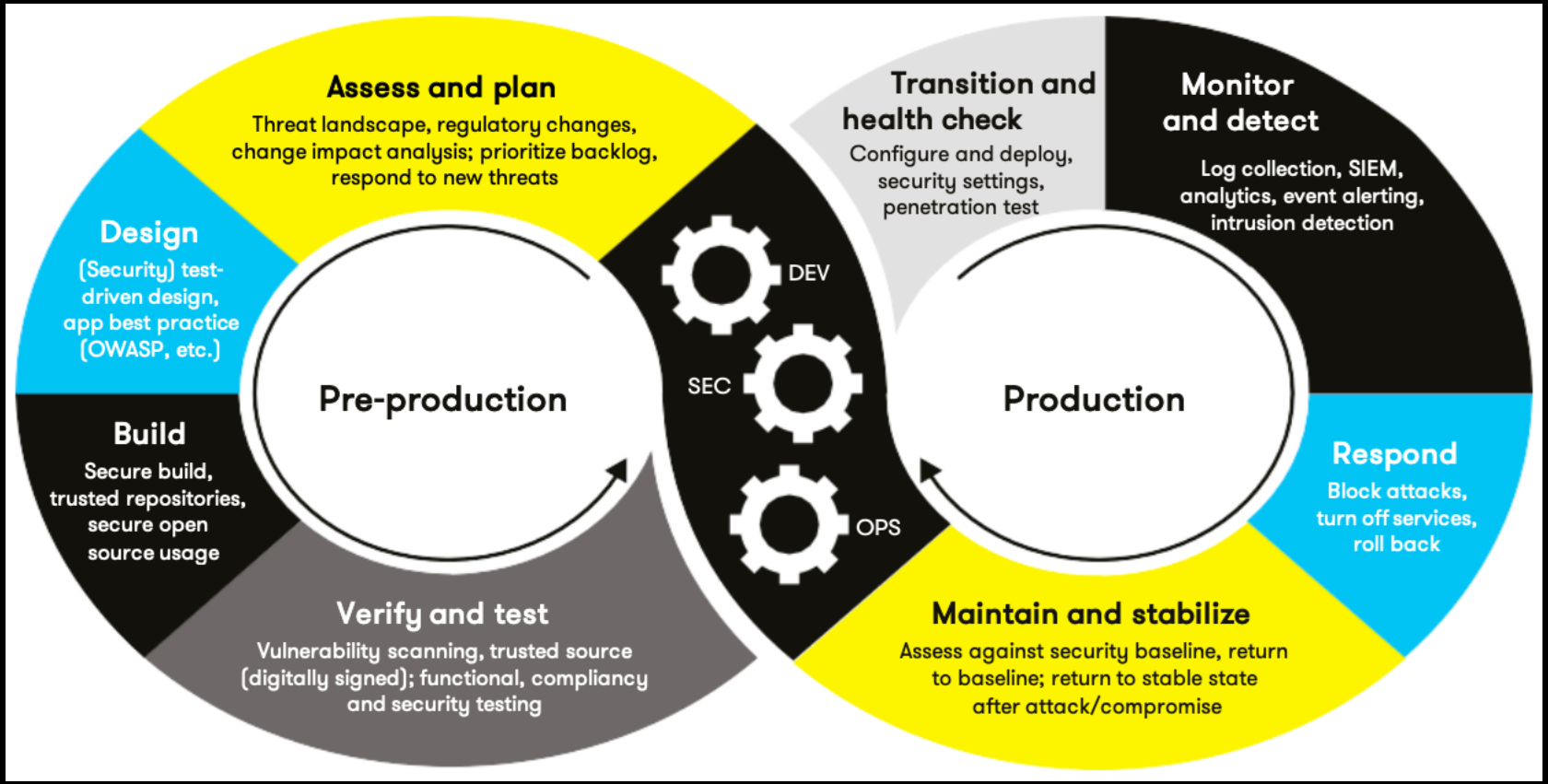
- Prevents invalid or malicious data

How to improve

- Add boundary testing (min/max values)
- Add automated input fuzzing

Green Pace

AUTOMATION SUMMARY



Green Pace

TOOLS

Create Phase (Development)

- Visual Studio Analyzer
- Detects Coding Issues while writing code

Build Phase

- Compiler C++
- Identifies syntax errors and warnings

Verify Phasing (testing)

- CPPCheck
- SonarQube
- Clang Static Analyzer
- Detect Vulnerabilities (memory Issues, SQL injections, Unsafe inputs)

Pre-Release / Testing

- Unit Testing Framework (Visual Studio C++)
- Fuzz Testing
- Detect Runtime and edge case vulnerabilities

Release Phase

- Code Signing
- Integrity checks
- Ensures only secure code is deployed

Green Pace

RISKS AND BENEFITS

- ***Problem***
  - Security vulnerabilities in code (Like input validation, memory issues, and SQL Injection)
- ***Solution***
  - Apply Secure coding stands
  - Use Automation tools and unit testing

- ***Act NOW!***
  - Prevents data breaches
  - Refuses risk of attacks
  - Improves system Reliability
  - Save cost long term

- ***Delay fixing***
  - Higher Chance of security breachers
  - Data loss or system compromise
  - Increased repair cost later
  - Damage to company reputation

Green Pace

RECOMMENDATIONS

Current Gaps

- Limited real-time threat detection
- Inconsistent automated security coverage across all stages
- Some manual dependency in testing and reviews

Recommendations

- Expand automated security tools in CI/CD pipeline
- Increase fuzz testing for edge-case detection
- Enforce stricter secure coding reviews before merging code
- Integrate continuous monitoring and logging alerts
- Regularly update security standards based on new threats

Green Pace

CONCLUSIONS

- Security is something that is continuous process ***not*** a 1-time task
- Defense in depth reduces risk though multiple layers of protection
- Secure Coding Standards prevent vulnerabilities
- Automation and test improve detection and response time.

Green Pace

REFERENCES

- *27 biggest data breaches globally (+ lessons) 2025. Huntress. (n.d.). https://www.huntress.com/blog/biggest-data-breaches GeeksforGeeks. (2025, December 17). Defence in depth.*
- *https://www.geeksforgeeks.org/ethical-hacking/defence-in-depth/ Significant cyber incidents: Strategic technologies program. CSIS. (n.d.).*
- *https://www.csis.org/programs/strategic-technologies-program/significant-cyber-incidents Top 20 most common types of cyber attacks. Fortinet. (n.d.).*
- *https://www.fortinet.com/resources/cyberglossary/types-of-cyber-attacks What is cybersecurity?. Cisco. (2025, July 15).*
- *https://www.cisco.com/site/us/en/learn/topics/security/what-is-cybersecurity.html What is defense in depth? | layered security | cloudflare. (n.d.).*
- *https://www.cloudflare.com/learning/security/glossary/what-is-defense-in-depth/*

Green Pace